

Overall Design of Program

The Universal Machine (UM) implementation is designed with modularity and clean separation of concerns as its guiding principles. The architecture breaks down the UM into distinct components, each with a clear responsibility and a well-defined API, allowing for independent development and testing. The core components include execution, loading, and memory management, which interact through interfaces that hide the underlying details and maintain a high level of abstraction. This design enables efficient unit testing and future extensibility. Segmented memory, a central aspect of the UM, is treated as a separate module to encapsulate the complexity of memory operations, allowing the UM to simulate a realistic machine environment with multiple segments and a dynamic mapping system.

Representation of Segments and Invariants

Segments:

- Represented as arrays or vectors of 32-bit words.
- Memory is divided into segments, each with their own 32-bit identifier.
- Each segment contains a sequence of words.
- Segments are dynamically allocated and deallocated to simulate memory mapping and unmapping.
- Segment zero always contains the program being executed.

Invariants:

- A unique identifier is associated with each active segment.
- Unmapped identifiers can be reused for new segments.
- Segments must maintain their size and content until explicitly modified by machine instructions.
- Segment Identifiers are unique and only reused when a segment is unmapped.
- The segment with the identifier 0 (or '\$m[0]') is reserved for the program being executed

Architecture

- The UM architecture will consist of separate components for execution, loading, and the main driver.
- Execution and loading components will each have their own API for modularity.
- Segmented memory will be managed by a dedicated module, ensuring encapsulation of memory operations.

Universal Machine

1. General-Purpose Registers

- a. Array of 32-bit integers (to store the words)
- 2. Program Counter**
 - a. 32-bit counter for tracking instruction addresses
- 3. Memory Management Module**
 - a. Manages memory segments
 - b. Interacts with the Instructions Decoder for getting instructions
- 4. Input/Output Handler**
 - a. Manages input and output operations
- 5. Instructions Decoder and Execution**
 - a. Decoder**
 - i. Decodes the opcode from the instruction word
 - ii. Identifies the operation using the 14 preset instruction codes
 - b. Execution**
 - i. Executes instructions based on the decoded opcode
 - ii. Interacts with:
 - 1. General-Purpose Registers for data manipulation.
 - 2. Memory Management Module for instructions involving memory.
 - 3. Program Counter for selecting instruction operations
 - 4. Input/Output Handler for operations involving input and output

Testing Plan

***Test Cases - Should Work**

***Failure Modes - Should Fail** (unchecked runtime error)

- Unit tests for each component of the UM:
- Testing the integrity of segment mapping and unmapping.
- Verifying correct execution of each instruction type.
- Ensuring proper behavior of the program counter within segment zero.
- Integration tests to ensure components interact correctly when combined.
- Stress tests with complex programs to evaluate overall UM stability and performance.

1. Memory Management Module

Test Cases:

- a. Segment Mapping
 - i. Test adding new segments and ensure they are correctly mapped and accessible.
 - ii. Verify that segment identifiers are unique and properly assigned.
- b. Segment Unmapping
 - i. Verify that an unmapped segment is no longer accessible.
 - ii. Verify that the identifier of the unmapped segment is reusable.

Failure Modes:

- c. Attempt to access a segment that is not mapped.
- d. Attempt to unmap segment 0.
- e. Attempt to unmap a segment that has not yet been mapped.

2. Program Counter Module

Test Cases:

- a. Verify that the program counter increments correctly after each instruction.
- b. Verify behavior when the counter reaches the end of segment 0.

Failure Modes:

- c. Set the program counter to point outside the bounds of segment 0.

3. Instructions Decoder Module

Test Cases:

- a. Verify that a valid set of instructions is decoded properly.

Failure Modes:

- b. Attempt to feed a value that does not code for a valid instruction.

4. Arithmetic/Logic Operations

Test Cases:

- a. Verify that the arithmetic operations are executed correctly.
 - i. Addition
 - ii. Multiplication
 - iii. Division
- b. Verify that the logical operations are executed correctly.
 - i. Bitwise NAND
- c. Check a variety of inputs including edge cases for both operation types.

Failure Modes:

- d. Attempt to divide by 0.

5. Input/Output Operations Handler

Test Cases:

- a. Verify input/output functionality using a range of ASCII characters.
 - i. “/dev/urandom” contains special inputs that can be tested.
 - ii. ‘dd’ and ‘cmp’ commands should be used to make testing easier.
- b. Verify correct behavior for end-of-input conditions.

Failure Modes:

- c. Attempt to output a value larger than 255.

6. Segmented Load/Store Operations

Test Cases:

- a. Verify the correct execution when loading and storing data into a variety of segments and positions.

Failure Modes:

- b. Attempt to load/store data from an unmapped segment.
- c. Attempt to load/store data outside the bounds of a mapped segment.

7. Load Program Operation

Test Cases:

- a. Verify that after duplicating a segment, the program counter is correctly set to the new segments start.

Failure Modes:

- b. Attempt to load a program from an unmapped segment.

Components/Modules

1. UM Execution Component:

Purpose:

- To serve as the central processing unit of the Universal Machine, executing instructions according to the UM specification.

Design:

- This component will have its own API to fetch, decode, and execute instructions, handling all computational and control flow operations.
- It will interact with the general-purpose registers for arithmetic operations and with the memory management module for memory-related instructions.
- The instruction decoder will be part of this component, decoding instructions and directing the flow of execution accordingly.

Testing Plan:

- Unit tests will be designed to check the execution of each type of instruction and its effects on the machine's state.
- Tests will include common scenarios as well as edge cases, such as overflow or underflow conditions in arithmetic operations.
- Fault injection testing for handling incorrect instructions or invalid states to ensure graceful failure or appropriate error messages.

2. UM Loading Component:

Purpose:

- To load UM programs from files and initialize the machine's state, particularly segment zero.

Design:

- The loader will parse program files and set up the initial memory segments, including the segment zero with the program's code.
- It will expose an API to accept file paths, read the contents, and process them into the UM's memory structure.

Testing Plan:

- Verify that after duplicating a segment, the program counter is correctly set to the new segments start.
- Unit tests to verify that files are correctly parsed and loaded into memory segments.
- Validation tests to ensure that the program counter starts at the correct location after loading.
- Error testing for file read issues, format errors, and handling non-executable files.

3. Main() Function/Module:

Purpose:

- To serve as the entry point of the UM program, orchestrating the setup and execution process.

Design:

- The main() function will be minimal, handling command-line arguments and leveraging the loading and execution modules to start the UM.
- It will act as the glue code, stitching together the components for a complete runtime environment.

Testing Plan:

- Integration tests could verify that main() correctly initializes the UM and starts execution with appropriate inputs.
- System tests to confirm that the UM behaves as expected when run from end to end.

4. Memory Management Module:

Purpose:

- The UM Memory Management Module is responsible for abstracting the details of the UM's memory architecture, providing a layer through which the UM execution module can access and manipulate memory segments.

Design:

- This module will provide an API for segment mapping, unmapping, and accessing.
- Will provide an API for memory operations required by UM execution, such as loading values from and storing values to memory.
- Ensures that each memory segment's integrity is maintained and provides mechanisms for dynamic memory allocation and deallocation.

Testing Plan:

- Unit tests to verify segment mapping and unmapping, including reuse of segment identifiers.
- Tests to check for proper access controls, ensuring unmapped segments cannot be accessed and that segmentation faults are handled.

5. On-Disk Isolation Module

Purpose:

- To abstract the specifics of the UM's on-disk program format and provide a consistent interface for loading programs into memory.

Design:

- This module will define the on-disk format of UM programs and be the only component that knows how to interpret this format.
- It will provide an API to convert from the on-disk format to the in-memory format required by the execution module.
- The design will ensure that changes to the on-disk format do not affect other components of the system, adhering to the principle of encapsulation.

Testing Plan:

- Unit tests to validate the correct interpretation of the on-disk format, including handling of various edge cases and format anomalies.
- Tests to ensure that the module correctly handles invalid or corrupted on-disk formats, reporting errors as necessary.
- Integration tests with the loading module to ensure that programs are correctly loaded into memory and ready for execution by the UM.

6. Bitpack Module (From A4 Arith):

Purpose:

- The bitpack module provides a set of utilities for bit manipulation, which is fundamental for the operation of the UM. These utilities allow the UM to handle word-oriented operations that involve packing and unpacking of bits, as well as checking the fit of values within a given number of bits, which is crucial for the UM's instruction decoding and memory management.

Design:

- Bitpack contains functions to determine if signed (`fitss`) and unsigned (`fitsu`) integers can fit within a specified number of bits, essential for ensuring data integrity when storing and retrieving values in the UM's registers and memory segments.
- The `gets` and `getu` functions retrieve signed and unsigned values, respectively, from a specified field within a word, using bit masking and shifting techniques. These functions are integral to the UM's execution engine when it decodes instructions and extracts operands.
- The `newu` and `news` functions are designed to insert unsigned and signed values into a word, which are used during instruction encoding and memory updates.

- Each function performs necessary checks to ensure that operations do not exceed the word size and that values fit within the specified widths, safeguarding against errors that could corrupt the UM's state.

Testing Plan:

- Unit tests for `fitss` and `fitsu` cover a range of inputs to ensure that the functions correctly identify when integers do or do not fit within the specified width.
- Unit tests for `gets` and `getu` validate correct retrieval of values across different field widths and positions within the word, including edge cases at the boundaries of the word.
- The `newu` and `news` functions are tested to verify that they correctly insert values into a word without altering unintended bits, and that they return `None` when a value does not fit or parameters are invalid.
- Additional tests will handle cases where inputs are on the boundaries of the word size, such as maximum and minimum values for signed and unsigned fields, and confirm proper error handling, such as panicking on invalid inputs.
- Integration tests will be developed to ensure that bitpack functions interact correctly with other UM components, such as the execution engine and memory management module, particularly in the context of instruction decoding and memory segment operations.

Design Considerations for Segments

What happens if I make several calls to map segment, then unmap one of the earlier calls, then make more calls to map segment?

- When you make several calls to map segments, the memory management module should:
 - Allocate a new memory segment for each call.
 - Assign a unique identifier to each new segment.
 - Store the content and size of each segment, initializing them as required.
 - Keep track of the mapped segments in a data structure, typically a table or array.
- When you unmap one of the earlier mapped segments, the memory management module should:
 - Mark the segment's identifier as unmapped in the tracking structure.
 - Deallocate the memory associated with that segment.
 - Optionally, the identifier of the unmapped segment can be added to a pool of available identifiers for reuse.

- After unmapping a segment, if you make more calls to map new segments, the memory management module should:
 - Check the pool of available identifiers first to reuse them for new segments.
 - If the pool is empty, it should continue to assign new unique identifiers.
 - Allocate memory for the new segments and initialize them as required.
 - Update the tracking structure to reflect the new mappings.

Segmented Memory Representation Invariant

- Each mapped segment must have a unique identifier that is not shared with any other segment, whether active or inactive
- The data within a mapped segment must be accessible and correctly represent the state of that segment. Upon unmapping, the data associated with a segment becomes inaccessible
- The mapping operation must always result in a valid segment allocation. If there are available identifiers from previously unmapped segments, these should be used first. New identifiers are only generated if no reusable identifiers exist
- The system must keep a clear and current record of which segments are mapped and which are not. The state must reflect all mapping and unmapping operations that have been executed
- Once a segment is unmapped, its identifier is either marked as reusable or destroyed. If reusable, it should be available for allocation to a new segment
- The identifiers for segments must be within the allowable range of the system. All segment operations must only be performed on valid identifiers that correspond to currently mapped segments
- Segment zero, typically containing the program itself, is always mapped and cannot be unmapped. Attempts to unmap segment zero should be handled as an error
- Unmapped segments must release their associated memory back to the system. There should be no memory leaks (unreleased memory) or overlaps (two segments sharing the same memory space)
- Any operation that violates the invariant (such as accessing an unmapped segment) should result in a well-defined error state or exception, consistent with the design of the UM's error handling strategy